Introduction to Transaction Concurrency 🤝

Transaction concurrency is the ability of a database management system (DBMS) to execute multiple transactions simultaneously. It's a critical concept for modern database systems, as it improves system throughput and resource utilization, especially in multi-user environments. Concurrency is essential for applications that handle a high volume of transactions at the same time, such as e-commerce platforms and banking systems.

The core idea is to allow transactions to overlap in their execution without compromising the integrity of the data. This is a complex task because concurrent access can lead to various problems if not managed correctly.

Concurrency Problems

Uncontrolled concurrent access can lead to data inconsistency and corruption. Here are the most common problems:

1. The Lost Update Problem 💥

This problem occurs when two or more transactions read the same data and then update it, and the update of one transaction overwrites the update of another. The changes made by the first transaction are "lost."

- **Example**:

1. **T1** reads a bank balance of $100.
2. **T2** reads the same balance of $100.
3. **T1** adds $50 (balance becomes $150) but hasn't committed the change yet.
4. **T2** adds $20 (balance becomes $120) and commits the change.
5. **T1** commits its change, writing $150 to the database.

- The final balance is $150, but the $20 addition from **T2** is lost.

2. The Dirty Read Problem (or Uncommitted Dependency) 📝

A dirty read occurs when one transaction reads data that has been modified by another concurrent transaction that has not yet been committed. If the second transaction is later rolled back, the data read by the first transaction is invalid.

- **Example**:

1. **T1** updates an item's quantity to 50 but hasn't committed.
2. **T2** reads the new quantity (50).
3. **T1** fails and is rolled back. The item's quantity reverts to its original value, say 100.

- **T2** made a decision based on an incorrect quantity (50), leading to an inconsistent state. The data it read was "dirty."

3. The Non-Repeatable Read Problem 🔄

This problem occurs when a transaction reads the same data twice, but another transaction modifies that data between the two reads. The first transaction gets different results for the same query.

- **Example**:
1. **T1** reads a student's grade, which is currently 'A'.
2. **T2** updates the student's grade to 'B' and commits the change.
3. **T1** reads the student's grade again and now sees 'B'.
- The second read yields a different result, violating the assumption that a transaction should see a consistent state of data.

4. The Phantom Read Problem 👻

A phantom read occurs when a transaction reads a set of rows that satisfy a certain condition, and a second transaction inserts new rows that also satisfy that condition. When the first transaction re-executes its query, it finds new "phantom" rows.

- **Example**:
1. **T1** counts the number of students from "Computer Science." It finds 100 students.
2. **T2** inserts a new student into the "Computer Science" department and commits.
3. **T1** counts the number of "Computer Science" students again and now finds 101.
- The new row, which was not there in the first read, appears like a "phantom."

Concurrency Control Mechanisms

To prevent these problems, DBMSs use **concurrency control mechanisms**. These techniques ensure that transactions are executed in an isolated and serializable manner, even if they run concurrently.

- **Locking**: This is the most common method. Transactions lock data items they are accessing to prevent other transactions from modifying them. There are different types of locks, such as shared locks (for reading) and exclusive locks (for writing).
- **Timestamp-Based Protocols**: Each transaction is assigned a unique timestamp. The DBMS uses these timestamps to determine the serial order of transactions and to manage conflicts.
- **Validation-Based Protocols**: Transactions are allowed to run freely. Before committing, the DBMS checks to see if the transaction's changes conflict with other transactions. If a conflict is found, the transaction is aborted.